第 1 章 基础算法（上）

💡 **小提醒：**

- 学习本章之前要有一个认知，**基础 ≠ 简单，基础 ≠ 简单，基础 ≠ 简单！**
- 基础说明这个算法适用性比较广，**是我们学习较难算法的基石**。但是不代表它所对应的题目就是简单题。
- 因此学习的时候，不要轻视基础算法，**它是你能爬的更高的根本**。

1. 模拟

模拟，顾名思义，就是题目让你做什么你就做什么，考察的是**将思路转化成代码的代码能力**。

这类题一般较为简单，属于竞赛里面的**签到题**（但是，万事无绝对，也有可能会出现让人非常难受的模拟题），我们在学习语法阶段接触的题，大多数都属于模拟题。

1.1 多项式输出

> **题目来源：** 洛谷
>
> **题目链接：** P1067 [NOIP2009 普及组] 多项式输出
>
> **难度系数：** ★

【题目描述】

> 一元 n 次多项式可用如下的表达式表示：
>
> $$f(x)=a_nx^n+a_{n-1}x^{n-1}+...+a_1x+a_0,a_0\neq 0$$
>
> 其中，a_ix^i 称为 i 次项，a^i 称为 i 次项的系数。给出一个一元多项式各项的次数和系数，请按照如下规定的格式要求输出该多项式：
>
> 1. 多项式中自变量为 x，从左到右按照次数递减顺序给出多项式。
> 2. 多项式中只包含系数不为 0 的项。
> 3. 如果多项式 n 次项系数为正，则多项式开头不出现 “+” 号，如果多项式 n 次项系数为负，则多项式以 “-” 号开头。

4. 对于不是最高次的项，以 “+” 号或者 “-” 号连接此项与前一项，分别表示此项系数为正或者系数为负。紧跟一个正整数，表示此项系数的绝对值（如果一个高于 0 次的项，其系数的绝对值为 1 ，则无需输出 1 ）。如果 x 的指数大于 1 ，则接下来紧跟的指数部分的形式为 “x^b”，其中 b 为 x 的指数；如果 x 的指数为 1 ，则接下来紧跟的指数部分形式为 “x”；如果 x 的指数为 0 ，则仅需输出系数即可。
5. 多项式中，多项式的开头、结尾不含多余的空格。

【输入描述】

- 第一行 1 个整数， n ，表示一元多项式的次数。
- 第二行有 $n+1$ 个整数，其中第 i 个整数表示第 $n-i+1$ 次项的系数，每两个整数之间用空格隔开。

【输出描述】

共 1 行，按题目所述格式输出多项式。

【示例一】

输入：

```
5
100 -1 1 -3 0 10
```

输出：

```
100x^5-x^4+x^3-3x^2+10
```

【示例二】

输入：

```
3
-50 0 0 1
```

输出：

```
-50x^3+1
```

【解法】

模拟 + 分类讨论，对于一元 n 次方程的的最终结果，我们仅需按照顺序，考虑每一项的三件事情：符号 + 系数 + 次数。

- 处理「符号」：
  - 如果系数小于 0 ，直接输出 "-"；
  - 如果系数大于 0 ，除了首项不输出 "+"，其余全部输出 "+"。
- 处理「系数」：
  - 先取一个绝对值，因为正负的问题已经处理过了；

- 当系数不等于 1 ，直接输出这个数；
- 但是当系数为 1 ，且是最后一项的时候，这个 1 也是需要输出的；其余情况下的 1 不需要输出。

- 处理「次数」：
  - 次数大于 1 ，输出 "x^" + 对应的次数；
  - 次数等于 1 ，输出 "x"；
  - 次数小于 1 ，什么也不输出。

【参考代码】

代码块

```cpp
#include <iostream>
#include <cmath>

using namespace std;

int main()
{
    int n; cin >> n;

    // 循环次数
    for(int i = n; i >= 0; i--)
    {
        int a; cin >> a;
        if(a == 0) continue; // 处理系数为 0 的情况

        // 1. 符号
        if(a < 0) cout << '-';
        else
        {
            if(i != n) cout << '+';
        }

        // 2. 数字
        a = abs(a);
        if(a != 1 || (a == 1 && i == 0)) cout << a;

        // 3. 次数
        if(i == 0) continue;
        else if(i == 1) cout << 'x';
        else cout << "x^" << i;

```

```
        }

        return 0;
    }
```

1.2 蛇形方阵

> **题目来源：** 洛谷
>
> **题目链接：** P5731 【深基5.习6】蛇形方阵
>
> **难度系数：** ★

【题目描述】

> 给出一个不大于 9 的正整数 n，输出 $n \times n$ 的蛇形方阵。
>
> 从左上角填上 1 开始，顺时针方向依次填入数字，如同样例所示。注意每个数字有都会占用 3 个字符，前面使用空格补齐。

【输入描述】

> 输入一个正整数 n，含义如题所述。

【输出描述】

> 输出符合题目要求的蛇形矩阵。

【示例一】

> 输入：
>
> ```
> 4
> ```
>
> 输出：
>
> ```
>  1  2  3  4
> 12 13 14  5
> 11 16 15  6
> 10  9  8  7
> ```

【解法】

模拟填数的过程。

在一个矩阵中按照一定规律填数的通用解法：

- 定义方向向量，比如本题一共四个方向，分别是右、下、左、上，对应：$(0,1)$、$(1,0)$、$(0,-1)$、$(-1,0)$
- 循环填数的规则：

- 朝一个方向走，一边走一边填数，直到越界；
- 越界之后，结合定义的方向向量，求出下一轮应该走的方向以及应该到达的正确位置；
- 重复上述过程，直到把所有的数填完为止。

【参考代码】

代码块

```
#include <iostream>

using namespace std;

const int N = 15;

// 定义 右，下，左，上 四个方向
int dx[] = {0, 1, 0, -1};
int dy[] = {1, 0, -1, 0};

int arr[N][N];

int main()
{
    int n; cin >> n;

    // 模拟填数过程
    int x = 1, y = 1; // 初始位置
    int cnt = 1; // 当前位置要填的数
    int pos = 0; // 当前的方向

    while(cnt <= n * n)
    {
        arr[x][y] = cnt;

        // 计算下一个位置
        int a = x + dx[pos], b = y + dy[pos];

        // 判断是否越界
        if(a < 1 || a > n || b < 1 || b > n || arr[a][b])
        {
            // 更新出正确的该走的位置
            pos = (pos + 1) % 4;
            a = x + dx[pos], b = y + dy[pos];
        }

        x = a, y = b;
```

```
            cnt++;
        }

        // 输出
        for(int i = 1; i <= n; i++)
        {
            for(int j = 1; j <= n; j++)
            {
                printf("%3d", arr[i][j]);
            }
            puts("");
        }


        return 0;
    }
```

1.3 字符串的展开

题目来源： 洛谷

题目链接： P1098 [NOIP2007 提高组] 字符串的展开

难度系数： ★

【题目描述】

在初赛普及组的“阅读程序写结果”的问题中，我们曾给出一个字符串展开的例子：如果在输入的字符串中，含有类似于 “d-h” 或 “4-8” 的子串，我们就把它当作一种简写，输出时，用连续递增的字母或数字串替代其中的减号，即，将上面两个子串分别输出为 “defgh” 和 “45678” 。在本题中，我们通过增加一些参数的设置，使字符串的展开更为灵活。具体约定如下：

（1）遇到下面的情况需要做字符串的展开：在输入的字符串中，出现了减号 “-” ，减号两侧同为小写字母或同为数字，且按照 ASCII码 的顺序，减号右边的字符严格大于左边的字符。

（2）参数 p_1 ：展开方式。 $p_1 = 1$ 时，对于字母子串，填充小写字母； $p_1 = 2$ 时，对于字母子串，填充大写字母。这两种情况下数字子串的填充方式相同。 $p_1 = 3$ 时，不论是字母子串还是数字子串，都用与要填充的字母个数相同的星号 “*” 来填充。

（3）参数 p_2 ：填充字符的重复个数。 $p_2 = k$ 表示同一个字符要连续填充 k 个。例如，当 $p_2 = 3$ 时，子串 “d-h” 应扩展为 “deeefffgggh” 。减号两侧的字符不变。

（4）参数 p_3 ：是否改为逆序： $p_3 = 1$ 表示维持原有顺序， $p_3 = 2$ 表示采用逆序输出，注意这时仍然不包括减号两端的字符。例如当 $p_1 = 1$、$p_2 = 2$、$p_3 = 2$ 时，子串 “d-h” 应扩展为 “dggffeeh” 。

> （5）如果减号右边的字符恰好是左边字符的后继，只删除中间的减号，例如：“d-e” 应输出为 “de”，“3-4” 应输出为 “34”。如果减号右边的字符按照 ASCII 码的顺序小于或等于左边字符，输出时，要保留中间的减号，例如：“d-d” 应输出为 “d-d”，“3-1” 应输出为 “3-1”。

【输入描述】

> 第 1 行为用空格隔开的 3 个正整数，依次表示参数 p_1, p_2, p_3。
> 第 2 行为一行字符串，仅由数字、小写字母和减号 “-” 组成。行首和行末均无空格。

【输出描述】

> 输出一行，为展开后的字符串。

【示例一】

> 输入：
>
> 1 2 1
>
> abcs-w1234-9s-4zz
>
> 输出：
>
> abcsttuuvvw1234556677889s-4zz

【示例二】

> 输入：
>
> 2 3 2
>
> a-d-d
>
> 输出：
>
> aCCCBBBd-d

【示例三】

> 输入：
>
> 3 4 2
>
> di-jkstra2-6
>
> 输出：
>
> dijkstra2**********6

【解法】

根据规则模拟字符串展开的过程即可。

题目的规则较长，但是叙述的非常详细，只要把「细节」考虑到位，「分类讨论」就能搞定。

【参考代码】

代码块

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

int p1, p2, p3, n;
string s;
string ret;

// 判断是否是数字字符
bool isdig(char ch)
{
    return ch >= '0' && ch <= '9';
}

// 判断是否是小写字母
bool islet(char ch)
{
    return ch >= 'a' && ch <= 'z';
}

// 把 [left, right] 之间的字符展开
// left, right 这两个字符是不做处理
void add(char left, char right)
{
    string t;

    // 遍历中间的字符
    for(char ch = left + 1; ch < right; ch++)
    {
        char tmp = ch;
        // 处理 p1
        if(p1 == 2 && islet(tmp)) tmp -= 32; // 小写变大写
        else if(p1 == 3) tmp = '*'; // 变成星号

        // 处理 p2
        for(int i = 0; i < p2; i++)
        {
            t += tmp;
        }
    }

    // 处理 p3
    if(p3 == 2) reverse(t.begin(), t.end());

    ret += t;
}
```

```
48
49  int main()
50  {
51      cin >> p1 >> p2 >> p3 >> s;
52      n = s.size();
53
54      for(int i = 0; i < n; i++)
55      {
56          char ch = s[i];
57
58          if(s[i] != '-' || i == 0 || i == n - 1) ret += ch;
59          else
60          {
61              char left = s[i - 1], right = s[i + 1];
62              // 判断是否展开
63              if(isdig(left) && isdig(right) && right > left ||
64                 islet(left) && islet(right) && right > left)
65              {
66                  // 展开
67                  add(left, right);
68              }
69              else
70              {
71                  ret += ch;
72              }
73          }
74      }
75
76      cout << ret << endl;
77
78      return 0;
79  }
```

2. 高精度

当数据的值特别大，各种类型都存不下的时候，此时就要用高精度算法来计算加减乘除：

- 先用字符串读入这个数，然后用数组逆序存储该数的每一位；
- 利用数组，模拟加减乘除运算的过程。

高精度算法本质上还是模拟算法，用代码模拟小学列竖式计算加减乘除的过程。

2.1 高精度加法